# M2: Mid-submission – Green Home Score & Retrofit Recommendation Engine

Megathon X: Singularity | Problem Statement 2 (Lloyds) | **Team: Commit(ment)-Issues** | **Team No. 78**

## 1 Current approach and changes since M1

The architecture of M1 is unchanged: a deterministic engine (`scenario_builder.build`) is the *only* writer of numbers; ML models plug in as services with provenance *modelled*; the LLM only explains engine results and turns what-if questions into validated engine requests. Changes since M1, each recorded as a numbered decision:

- **Engine v2/v3.** Segment-aware NEED impacts (empirical-Bayes shrinkage by EPC band, age, type, region); savings for electrically heated homes from NEED benchmarks; a NEED-calibrated reference figure shown *next to* the engineering estimate for combined packages (answering the 5.6–13.8 pp gap found at M1); boiler efficiency from the EPC heating rating; field SCOP 2.80 (2.44–3.5); roof-type-aware loft rule (flat roof ⇒ no loft); **bills** at the Ofgem cap, **grants** (Boiler Upgrade Scheme), net cost, payback and an *illustrative* loan; 300-draw Monte Carlo ranges (P10–P90, P(bill falls)); portfolio ranking options.
- **ML layer (new).** Shared feature layer (one code path for homes, past certificates and sales; parsed EPC descriptions); four model families trained with Optuna (seed 42, deterministic LightGBM), gated on validation against simple baselines; models stored as LightGBM text + metadata; every model version enters the scenario id.
- **LLM layer integrated.** The LLM never sees a number: it writes `{placeholders}` that code fills from an address-free payload with a stable key per engine value. A rule checker drops bad sentences and patches gaps from a template; a new **number checker** matches every number in the final answer to an engine value (one regeneration, then template). What-ifs (measures, tariffs, grants, partial heat pump, own meter reading, ranges, budget) are validated by the engine's own `ScenarioRequest` schema; invalid requests are refused with the reason.
- **API.** FastAPI: `/homes/{id}/score`, `/scenario`, `/explain` (answer, cited values, scenario id, grounded), `/whatif`, `/portfolio`. Works without an LLM key (template explainer).

## 2 Experiments run so far and results

**ML models** (York; property-level splits; tuning and gates on *validation*, test split scored *once*):

| Model | Metric (test) | Model | Baseline(s) | $n$ | Shipped |
|---|---|---|---|---|---|
| EPC SAP score | MAE (SAP points) | **3.46** | 5.96 median / 4.48 ridge | 9,919 | yes |
| counterfactual pairs | MAE of ΔSAP, same home | **5.03** | 6.66 per-measure mean | 880 | |
| Property value | median abs. % error | **9.0%** | 16.7% area median / 10.5% hedonic | 619 | yes |
| 80% range | coverage (gate 75–85%) | **82.1%** | – | 619 | gate **pass** |
| Energy (NEED, 4M) | MAE gas / elec (kWh), val | 3,570 / 1,390 | 3,626 / 1,411 cell medians | 326k/405k | **fail** (<2% gain) |
| quantiles | P10–P90 coverage | 0.80 | pinball 1,146 vs 1,166 | 325k | yes |
| Recommender | NDCG@3 over feasible measures | **0.995** | 0.521 engine order / 0.981 prevalence | 7,840 | yes |

- **SAP model:** monotone in every "better state" input; on 10,000 homes no fabric, glazing, controls or PV upgrade lowers the modelled SAP. It powers the indicative "E → D" shown next to the score. A heat pump can *lower* SAP (electricity priced above gas), which the explanation states.
- **Value:** hedonic $\beta_{\text{SAP}} = +0.19\%$ of price per SAP point (sector-clustered bootstrap 0.11–0.27%, 99.6% positive). Demo home A: indicative +1.0% to +4.2%; reported as an association, not a causal effect.
- **Energy:** banded NEED features leave little signal beyond cell medians, so the gate fails and the engine keeps the benchmark; the quantile models ship and feed the Monte Carlo.
- **Recommender:** per-measure PR-AUC 0.90–1.00 vs 0.59–0.98 for prevalence (thermostat 0.51 vs 0.19); heat pump has 23 training labels, so rules only. Agreement means "consistent with the EPC assessor", not "feasible".
- **Weak slices** (flagged at >1.5× overall error): SAP at EPC F/G (MAE 9–10), value for flats (15%).

**LLM.** A held-out set of 53 questions (demo homes A/B/C and five test-split homes: the brief's four questions, what-ifs, missing and conflicting data, infeasible requests, electricity rising, value not estimated, out of scope, injection), run end to end through the API on the real engine:

| Run | Grounded | Factual acc. | Unsupported | What-if success | Refusals correct |
|---|---|---|---|---|---|
| Target | ≥90% | ≥90% | <5% | – | – |
| Template explainer | 100% (585) | 100% (263) | 0% | 100% (17) | 100% |
| Mocked LLM (scripted, incl. an invented number) | 100% | 100% | 0% | 100% | 100% |

Live smoke tests (Groq `gpt-oss-120b/20b`): 5 questions answered with every number grounded; the checker dropped 3–6 sentences per answer (wrong filter use, unknown keys, typed units) and patched them from the template. The first evaluation run found real bugs (fuel assignment in "20p electricity and 6p gas", missing payload values), fixed and disclosed.

## 3 Progress on testing

- **719 automated tests, all passing** (`make test`): hand-calculated demo homes to $10^{-6}$, reconciliation, monotonicity of every sub-score, feature contract (no label or address column can reach a model), privacy guard (address columns never loaded), metrics against hand values, ML services with tiny synthetic models (abstain paths, determinism, scenario id changes with a model version), API routes, number checker, what-if validation.
- **Leakage controls:** encoders and tuning on train only; value features from the certificate lodged *before* the sale; test scored once by a script that refuses to run twice; counterfactual pairs from held-out homes.
- **Fixed since M1:** three tests encoded loft insulation for demo A, whose certificate records a flat roof (infeasible under our own roof rule); updated by hand arithmetic, not by the engine.

## 4 Plan for the remaining time

1. **Real-LLM evaluation** on all 53 questions (free-tier daily token caps stopped the first run) and an LLM-judge sample for qualitative support; fix the wording gaps seen live (masked "#" copied into text, what-if answer not naming the changed assumption).
2. **Frontend dashboard:** score and sub-score contributions, feasible measures with "needs verification", priority vs install order, per-fuel savings (electricity rise shown in red), bills, grants, payback, ranges, indicative EPC and value range, and a what-if chat on `/explain` and `/whatif`.
3. **Integration:** engine, ML and LLM work is merged into `main` with this submission; next: a one-command demo (`make api` + dashboard) scripted on homes A, B and C.
4. **Final report:** methods, validation tables (val and test with $n$), responsible-AI notes (provenance, caveats, privacy), and the decision log.